

Chapitre 3 : Création de nos premières pages

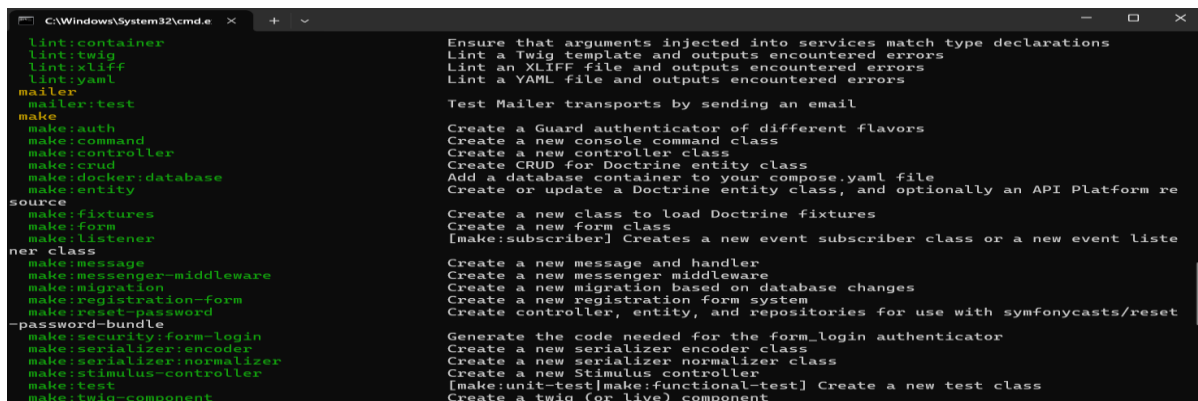
On va créer notre première page avec Symfony. Par défaut on arrivait sur la page d'accueil « **Welcome to Symfony 7** ». Pour changer ça on va devoir créer dans un premier temps un **Controller**. C'est une classe qui va contenir différentes méthodes, chaque méthode correspondra à une url particulière. On pourrait créer nous-mêmes ces classes mais Symfony permet de les créer automatiquement via la console (CMD).

3.1 La page index et les routes

Pour créer un **Controller**, il faut regarder les commandes

- **symfony console**

Cette commande vous liste toutes les commandes à votre disposition dans votre projet.



```
C:\Windows\System32\cmd.e x + v
lint:container      Ensure that arguments injected into services match type declarations
lint:twig           Lint a Twig template and outputs encountered errors
lint:xliff          Lint an XLIFF file and outputs encountered errors
lint:yaml           Lint a YAML file and outputs encountered errors
mailer             Test Mailer transports by sending an email
make               Create a Guard authenticator of different flavors
make:auth           Create a new console command class
make:command        Create a new controller class
make:controller     Create a new controller class
make:crud           Add a database container to your compose.yaml file
make:doctrine       Create or update a Doctrine entity class, and optionally an API Platform re
make:docker         Create a new class to load Doctrine fixtures
make:entity         Create a new form class
make:fixtures       [make:subscriber] Creates a new event subscriber class or a new event liste
make:form           Create a new message and handler
make:listener       Create a new messenger middleware
make:message        Create a new migration based on database changes
make:messenger-middleware Create a new registration form system
make:migration      Create a new registration form system
make:registration-form Create controller, entity, and repositories for use with symfonycasts/reset
make:reset-password Generate the code needed for the form_login authenticator
make:security:form-login Create a new serializer encoder class
make:serializer:encoder Create a new serializer normalizer class
make:serializer:normalizer Create a new Stimulus controller
make:stimulus-controller [make:unit-test|make:functional-test] Create a new test class
make:test           Create a twig (or live) component
make:twig-component
```

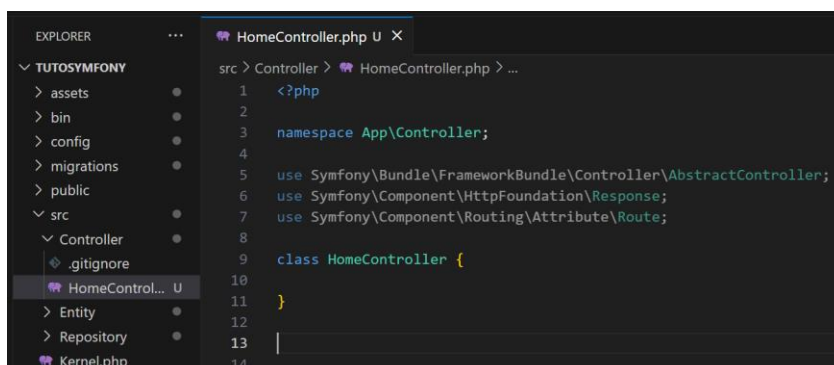
On verra qu'il y a un descriptif de chacune des commandes disponibles, on fera donc :

- **symfony console make:controller nomController**(pas obliger de noter Controller, il le rajoutera)

On va l'appeler **HomeController** ce qui donne donc **symfony console make:controller Home**.

Automatiquement il crée 2 fichiers, le **Controller HomeController.php** qui est une classe PHP dans le dossier **/src** et la vue (**View**) **index.html.twig** dans le dossier **/templates**.

Dans VScode allez là où se trouve votre Controller et ouvrez-le. Dans un premier temps on va supprimer ce qu'il y a dedans et vous aurez quelques choses comme ça :



```
EXPLORER
TUTOSYMFONY
  > assets
  > bin
  > config
  > migrations
  > public
  > src
    > Controller
      > HomeController.php
    > Entity
    > Repository
    > Kernel.php

src > Controller > HomeController.php > ...
1  <?php
2
3  namespace App\Controller;
4
5  use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
6  use Symfony\Component\HttpFoundation\Response;
7  use Symfony\Component\Routing\Attribute\Route;
8
9  class HomeController {
10
11  }
12
13
14
```

On va repartir d'une classe simple.

Un Controller, je rappelle que c'est tout simplement une classe qui va contenir des méthodes et chaque méthode va correspondre à une page.

On va créer notre première méthode qui sera l'index sans paramètres donc notre page d'accueil et non plus la page d'accueil Symfony.

Les méthodes dans les Controllers doivent toujours renvoyer une réponse (qui est un Objet qui provient de Symfony : **Response**). Attention celle qui vient de HttpFoundation.

Donc je vais retourner une Response, qui lorsqu'on le construit, reçoit en premier paramètre le contenu, on va mettre « **Hello World !!!** ».

- **return new Response("Hello World !!!") ;**

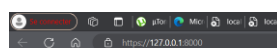
Maintenant il va falloir expliquer au Framework que l'URL racine va correspondre à cette méthode-là. Il y a deux manières de faire. Je vais vous montrer les deux bien que j'en utiliserai qu'une seule. Les deux sont importantes à savoir.

La première manière c'est d'aller dans le dossier config à la racine de votre projet, dans le fichier « **routes.yaml** ». Là on va donner le nom de notre route donc « **home** », lui donner son chemin avec « **path** » qui sera la racine donc « **/** ». On écrit ensuite « **controller** » à qui on donnera le nom de notre controller donc en précisant son namespace et la méthode qu'on veut exécuter qui est donc index « **App\Controller\HomeController::index** » :

```
config > ! routes.yaml
1  controllers:
2      resource:|
3          path: ../src/Controller/
4          namespace: App\Controller
5          type: attribute
6  home:
7      path: /
8      controller: App\Controller\HomeController::index
```

En gros j'explique à mon Framework que lorsqu'on a l'URL racine « **/** » il faut qu'il aille chercher cette action « **index** ». Attention dans YAML l'indentation est importante.

Une fois que c'est fait vous pourrez retourner dans votre navigateur et faire un refresh. Vous verrez votre Hello World !!! :



Hello World !!!

Voici la première manière de faire, qui était l'ancienne manière de faire. Beaucoup d'autres Frameworks utilisent ce système de mettre dans un fichier à part toutes les routes.